🛍️ Retail Funnel Intelligence: User Behavior Analysis

📌 Objective

This project analyzes 100K+ retail interaction events to understand user behavior, identify drop-off points in the conversion funnel, and uncover factors influencing purchase decisions.

Key Questions

- Where do users drop off in the funnel?
- What actions lead to higher conversions?
- How do device, channel, and product attributes impact purchases?
- Can we identify patterns in high-intent users?

In [1]:
```
import pandas as pd
import numpy as np
```

📥 Data Loading

We begin by loading the dataset and previewing its structure to understand available features.

In [2]:
```
df = pd.read_csv('retail_user_behavior_100k.csv')
df.head()
```

Out[2]:

	session_id	user_id	timestamp_utc	event_index	user_action	product_
0	S0000001	U000372	2026-01-08T02:34:40Z	1	view	P148
1	S0000001	U000372	2026-01-08T02:35:20Z	2	wishlist	P148
2	S0000001	U000372	2026-01-08T02:35:43Z	3	view	P148
3	S0000001	U000372	2026-01-08T02:36:13Z	4	drop	P148
4	S0000002	U004812	2026-01-29T11:07:27Z	1	view	P181

🔍 Data Overview

Understanding the structure, data types, and completeness of the dataset.

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 108584 entries, 0 to 108583
Data columns (total 18 columns):
 #   Column             Non-Null Count   Dtype  
---  ------             --------------   -----  
 0   session_id         108584 non-null  object 
 1   user_id            108584 non-null  object 
 2   timestamp_utc      108584 non-null  object 
 3   event_index        108584 non-null  int64  
 4   user_action        108584 non-null  object 
 5   product_id         108584 non-null  object 
 6   category           108584 non-null  object 
 7   brand              108584 non-null  object 
 8   price              108584 non-null  float64
 9   channel            108584 non-null  object 
 10  device_type        108584 non-null  object 
 11  region             108584 non-null  object 
 12  traffic_source     108584 non-null  object 
 13  time_spent_sec     108584 non-null  int64  
 14  session_length     108584 non-null  int64  
 15  interaction_count  108584 non-null  int64  
 16  is_conversion      108584 non-null  int64  
 17  drop_off_flag      108584 non-null  int64  
dtypes: float64(1), int64(6), object(11)
memory usage: 14.9+ MB
```

In [4]:
```
df.describe(include='all')
```

Out[4]:

	session_id	user_id	timestamp_utc	event_index	user_action	p
count	108584	108584	108584	108584.000000	108584	
unique	18000	6806	108020	NaN	6	
top	S0000924	U000344	2026-03-21T05:34:29Z	NaN	view	
freq	13	64	3	NaN	44245	
mean	NaN	NaN	NaN	3.834626	NaN	
std	NaN	NaN	NaN	2.238094	NaN	
min	NaN	NaN	NaN	1.000000	NaN	
25%	NaN	NaN	NaN	2.000000	NaN	
50%	NaN	NaN	NaN	4.000000	NaN	
75%	NaN	NaN	NaN	5.000000	NaN	
max	NaN	NaN	NaN	13.000000	NaN	

In [5]:
```
df.columns
```

Out[5]:
```
Index(['session_id', 'user_id', 'timestamp_utc', 'event_index', 'user_actio
n',
       'product_id', 'category', 'brand', 'price', 'channel', 'device_type',
       'region', 'traffic_source', 'time_spent_sec', 'session_length',
       'interaction_count', 'is_conversion', 'drop_off_flag'],
      dtype='object')
```

In [6]:
```
df.describe()
```

Out[6]:

	event_index	price	time_spent_sec	session_length	interactio
count	108584.000000	108584.000000	108584.000000	108584.000000	108584
mean	3.834626	249.497547	17.828317	6.669251	3
std	2.238094	145.627749	9.356853	2.038962	2
min	1.000000	7.290000	3.000000	3.000000	1
25%	2.000000	119.950000	11.000000	5.000000	2
50%	4.000000	253.850000	18.000000	7.000000	4
75%	5.000000	376.080000	24.000000	8.000000	5
max	13.000000	499.860000	64.000000	13.000000	13

🧹 Data Cleaning & Preparation

To ensure accurate analysis:

- Convert timestamps into proper datetime format
- Sort events to maintain session flow

In [7]:
```
df['timestamp_utc'] = pd.to_datetime(df['timestamp_utc'])

df = df.sort_values(by=['session_id', 'timestamp_utc'])
```

In [8]:
```
df.isnull().sum()
```

Out[8]:
```
session_id           0
user_id              0
timestamp_utc        0
event_index          0
user_action          0
product_id           0
category             0
brand                0
price                0
channel              0
device_type          0
region               0
traffic_source       0
time_spent_sec       0
session_length       0
interaction_count    0
is_conversion        0
drop_off_flag        0
dtype: int64
```

📊 User Action Distribution

Understanding how users interact with the platform by analyzing the frequency of each action type.

This helps identify:

- Most common user behaviors
- Potential drop-off stages
- Engagement patterns

In [9]:
```
df['user_action'].value_counts()
```

Out[9]:
```
user_action
view           44245
click          27735
drop           13797
add_to_cart    11642
wishlist        6962
purchase        4203
Name: count, dtype: int64
```

In [10]:
```
(df['user_action'].value_counts(normalize=True) * 100).round(2)
```

Out[10]:
```
user_action
view           40.75
click          25.54
drop           12.71
add_to_cart    10.72
wishlist        6.41
purchase        3.87
Name: proportion, dtype: float64
```

💡 Key Insight: Conversion Behavior

While add-to-cart represents strong purchase intent, only ~36% of those users complete the transaction, indicating significant friction in the checkout stage.

Additionally, a large drop-off occurs earlier in the funnel, suggesting challenges in converting product interest into buying intent.

🔄 Conversion Funnel Analysis

To better understand user behavior, we construct a conversion funnel representing the key stages:

View → Click → Add to Cart → Purchase

This helps identify:

- Conversion rates between stages
- Drop-off points in the user journey
- Areas where optimization is needed

In [11]:
```
funnel_counts = df['user_action'].value_counts()

funnel_counts = funnel_counts[['view', 'click', 'add_to_cart', 'purchase']]

funnel_counts
```

Out[11]:
```
user_action
view           44245
click          27735
add_to_cart    11642
purchase        4203
Name: count, dtype: int64
```

In [12]:
```
view_to_click = funnel_counts['click'] / funnel_counts['view']
click_to_cart = funnel_counts['add_to_cart'] / funnel_counts['click']
cart_to_purchase = funnel_counts['purchase'] / funnel_counts['add_to_cart']

print("View → Click:", round(view_to_click * 100, 2), "%")
```

```
print("Click → Add to Cart:", round(click_to_cart * 100, 2), "%")
print("Add to Cart → Purchase:", round(cart_to_purchase * 100, 2), "%")
```

```
View → Click: 62.69 %
Click → Add to Cart: 41.98 %
Add to Cart → Purchase: 36.1 %
```

In [13]:
```
funnel_df = pd.DataFrame({
    'Stage': ['View → Click', 'Click → Cart', 'Cart → Purchase'],
    'Conversion Rate (%)': [
        view_to_click * 100,
        click_to_cart * 100,
        cart_to_purchase * 100
    ]
})

funnel_df
```

Out[13]:

	Stage	Conversion Rate (%)
0	View → Click	62.685049
1	Click → Cart	41.975843
2	Cart → Purchase	36.102044

💡 Funnel Insights

- The highest drop-off occurs at the Click → Add-to-Cart stage (~42% conversion), indicating challenges in converting user interest into purchase intent.
- While Add-to-Cart represents strong intent, only ~36% of those users complete the purchase, highlighting friction in the checkout process.
- The View → Click stage shows relatively strong engagement (~63%), suggesting that users are interested but not sufficiently convinced to proceed further.

👤 Session-Level Behavior Analysis

While event-level analysis shows overall trends, session-level analysis helps us understand user behavior within individual sessions.

This allows us to answer:

- Do longer sessions lead to higher conversions?
- Do users with more interactions convert more?
- What differentiates converting vs non-converting sessions?

In [14]:
```
## Create Session-Level Dataset
session_df = df.groupby('session_id').agg({
    'user_id': 'first',
    'session_length': 'first',
    'interaction_count': 'first',
    'time_spent_sec': 'sum',
    'is_conversion': 'max'
}).reset_index()

session_df.head()
```

Out[14]:

	session_id	user_id	session_length	interaction_count	time_spent_sec	is_co
0	S0000001	U000372	4	1	74	
1	S0000002	U004812	4	1	60	
2	S0000003	U001935	6	1	122	
3	S0000004	U001996	3	1	60	
4	S0000005	U000024	4	1	65	

📊 Session Dataset Created

Each row now represents a unique session with aggregated behavioral features.

Compare Converting vs Non-Converting Users

In [15]:
```
session_df.groupby('is_conversion').mean(numeric_only=True)
```

Out[15]:

is_conversion	session_length	interaction_count	time_spent_sec
0	6.035877	1.0	107.496050
1	6.021175	1.0	107.719962

💡 Session-Level Insights

- There is minimal difference between converting and non-converting sessions in terms of session length, interaction count, and time spent.
- Engagement metrics alone do not strongly explain conversion behavior in this dataset.
- This suggests that factors such as user intent, product attributes, or pricing may play a more significant role in driving conversions.

⚠ Key Observation: Limitations of Basic Features

The initial session-level analysis reveals that basic engagement metrics such as session length, interaction count, and time spent do not significantly differ between converting and non-converting users.

This indicates that:

- These features have low discriminatory power for predicting conversions
- User engagement alone is not a reliable indicator of purchase behavior in this dataset

🔍 Implication

To better understand and predict user conversions, more granular behavioral features need to be engineered. This includes analyzing specific user actions (e.g., clicks, add-to-cart events) and their sequence within sessions.

➡ The next step focuses on feature engineering to extract more meaningful signals from user behavior.

🛠️ Feature Engineering

Since basic session-level features did not provide strong predictive signals, we now create more granular behavioral features based on user actions within each session.

These features aim to capture:

- User intent (e.g., add-to-cart actions)
- Engagement depth (e.g., number of clicks)
- Behavioral patterns leading to conversion

🧠 What We’re Doing

We’ll create features like:

- number of views per session
- number of clicks
- number of add-to-cart

- number of purchases
- number of drops

In [16]:
```
action_features = pd.crosstab(df['session_id'], df['user_action'])

action_features.head()
```

Out[16]:

user_action session_id	add_to_cart	click	drop	purchase	view	wishlist
S0000001	0	0	1	0	2	1
S0000002	1	0	1	0	2	0
S0000003	3	1	0	1	1	0
S0000004	0	0	1	0	2	0
S0000005	0	0	1	0	2	1

In [17]:
```
session_df = session_df.merge(action_features, on='session_id', how='left')

session_df.head()
```

Out[17]:

	session_id	user_id	session_length	interaction_count	time_spent_sec	is_co
0	S0000001	U000372	4	1	74	
1	S0000002	U004812	4	1	60	
2	S0000003	U001935	6	1	122	
3	S0000004	U001996	3	1	60	
4	S0000005	U000024	4	1	65	

📊 Enhanced Session Dataset

The dataset now includes detailed action-based features, providing deeper insight into user behavior within each session.

In [18]:
```
session_df = session_df.fillna(0)
```

In [19]:
```
session_df.groupby('is_conversion')[['view', 'click', 'add_to_cart', 'purchase
```

Out[19]:

is_conversion	view	click	add_to_cart	purchase
0	2.609408	1.615714	0.409944	0.0
1	1.961218	1.295027	1.424221	1.0

💡 Feature Engineering Insights

- The add-to-cart action is the strongest predictor of conversion, with converting sessions showing significantly higher values.
- Users who convert tend to have fewer views and clicks, indicating more decisive behavior.
- Non-converting users exhibit higher browsing activity but lower purchase intent.
- Behavioral signals derived from user actions provide much stronger predictive power than basic engagement metrics.

⚠ Preparing Data for Machine Learning

Before building a machine learning model, it is important to ensure that the dataset does not contain any **data leakage**.

🔍 What is Data Leakage?

Data leakage occurs when information that directly indicates the target variable is included as an input feature, leading to overly optimistic and misleading model performance.

🚨 In This Dataset

The `purchase` column directly represents whether a user converted within a session. Since our target variable is `is_conversion`, including `purchase` as a feature would result in leakage.

✅ Approach

- Create a separate dataset for machine learning
- Remove leakage-prone features such as `purchase`
- Retain only meaningful behavioral signals for prediction

➡ This ensures that the model learns genuine patterns rather than relying on direct

indicators of the outcome.

MACHINE LEARNING MODEL

🤖 Conversion Prediction Model

In this step, we build a machine learning model to predict whether a session will result in a conversion.

🎯 Objective

Predict `is_conversion` using behavioral features derived from user actions.

🧠 Approach

- Use session-level features
- Avoid data leakage by removing direct indicators like `purchase`
- Train a classification model to distinguish converting vs non-converting sessions

-- Create ML Dataset

In [20]:
```
ml_df = session_df.copy()
```

In [21]:
```
ml_df = ml_df.drop(columns=['purchase'])
```

In [22]:
```
X = ml_df.drop(columns=['is_conversion', 'session_id', 'user_id'])
y = ml_df['is_conversion']
```

In [23]:
```
## Train-Test Split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [24]:
```
#Train Model (Start Simple)
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Out[24]:

LogisticRegression

Parameters

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	1000
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

In [25]:
```
#Predictions
y_pred = model.predict(X_test)
```

In [26]:
```
# Evaluate Model

from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      2787
           1       1.00      1.00      1.00       813

    accuracy                           1.00      3600
   macro avg       1.00      1.00      1.00      3600
weighted avg       1.00      1.00      1.00      3600
```

⚠ Model Evaluation Warning: Potential Data Leakage

The model achieved near-perfect accuracy, which is highly unusual in real-world scenarios.

This suggests the presence of data leakage, where certain features (such as add-to-cart actions) are too closely tied to the target variable (`is_conversion`).

🔍 Issue

Features like `add_to_cart` may occur very close to the purchase event, making them strong proxies for conversion rather than true predictors.

⚠ Impact

- The model may not generalize well to real-world scenarios
- Performance metrics may be overly optimistic

🔍 Why Did the Model Perform Perfectly?

The model achieved 100% accuracy, which is highly unlikely in real-world scenarios. This suggests that the model is leveraging features that are too closely tied to the target variable.

🚨 Possible Reasons

- **Data Leakage**: Features like `add_to_cart` occur very close to the purchase event and act as strong proxies for conversion.
- **Highly Predictive Behavioral Signals**: Certain actions (e.g., add-to-cart) almost directly imply user intent to purchase.
- **Lack of Temporal Separation**: The model does not distinguish between early-stage behavior and actions that occur just before conversion.

🧠 Improving the Modeling Approach

To build a more realistic and generalizable model, the following strategies can be applied:

1️⃣ Remove High-Leakage Features

Exclude features that directly indicate purchase intent:

- `add_to_cart`
- `purchase`

Focus only on early-stage behavior like:

- views
- clicks

2️⃣ Use Early-Stage Data Only

Instead of using the full session, train the model on:

- First few interactions in a session
- Pre-cart behavior

This ensures the model predicts *future* conversion, not current intent.

3️⃣ Train-Test Split Improvements

Current approach uses a random split. Alternative approaches:

- **Session-based split (already used)** ✅
- **Time-based split (advanced)**
  Train on earlier data and test on later data to simulate real-world deployment.

4️⃣ Try Different Models

Instead of relying on a single model:

- Logistic Regression (baseline) ✅
- Random Forest (captures non-linearity)
- Gradient Boosting (advanced)

5⃣ Evaluate Beyond Accuracy

Accuracy alone can be misleading.

Use:

- Precision (how correct positive predictions are)
- Recall (how many conversions are captured)
- F1-score (balance of precision & recall)

🧪 Fixing the Model (Removing Leakage)

The initial model gave perfect accuracy, which is unrealistic and a clear sign of data leakage.

On closer inspection, features like `add_to_cart` were too closely tied to the final purchase, making the model's job too easy.

To fix this, I removed these high-intent features and rebuilt the model using only early-stage signals like views and clicks.

The goal here is to make the model more realistic — predicting conversion *before* strong buying intent is already visible.

In [27]:
```
ml_df_clean = session_df.copy()
```

In [28]:
```
## Drop Leakage Features
ml_df_clean = ml_df_clean.drop(columns=['purchase', 'add_to_cart'])
```

In [29]:
```
## Define Features & Target
X = ml_df_clean.drop(columns=['is_conversion', 'session_id', 'user_id'])
y = ml_df_clean['is_conversion']
```

In [30]:
```
## Train-Test Split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [31]:
```
## Train Model Again
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Out[31]:

LogisticRegression

Parameters

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	1000
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

In [32]:

```
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      2787
           1       1.00      1.00      1.00       813

    accuracy                           1.00      3600
   macro avg       1.00      1.00      1.00      3600
weighted avg       1.00      1.00      1.00      3600
```

🔍 Digging Deeper: Why Is Accuracy Still Perfect?

Even after removing high-intent features like `add_to_cart`, the model is still achieving 100% accuracy, which indicates that data leakage is still present.

This suggests that some of the remaining features may also be indirectly capturing information from the end of the session.

🚨 Possible Issue

Features such as:

- `session_length`
- `interaction_count`
- `time_spent_sec`

are likely calculated after the session is complete, making them post-event features.

As a result, the model is not truly predicting conversion — it is learning patterns that already reflect completed user behavior.

🎯 Next Step

To build a more realistic model, we will further restrict the feature set to only early-stage user actions, ensuring that predictions are based on signals available before strong purchase intent is observed.

In [33]:
```
ml_df_clean = session_df.copy()

ml_df_clean = ml_df_clean[[
    'view',
    'click',
    'is_conversion'
]]
```

In [34]:
```
X = ml_df_clean[['view', 'click']]
y = ml_df_clean['is_conversion']
```

In [35]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)

# train
model = LogisticRegression()
model.fit(X_train, y_train)
```

Out[35]:

LogisticRegression

Parameters

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	100
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

In [36]:

```
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7741666666666667
              precision    recall  f1-score   support

           0       0.77      1.00      0.87      2787
           1       0.00      0.00      0.00       813

    accuracy                           0.77      3600
   macro avg       0.39      0.50      0.44      3600
weighted avg       0.60      0.77      0.68      3600
```

```
C:\Users\shubh\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1
731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in l
abels with no predicted samples. Use `zero_division` parameter to control this
behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\shubh\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1
731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in l
abels with no predicted samples. Use `zero_division` parameter to control this
behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\shubh\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1
731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in l
abels with no predicted samples. Use `zero_division` parameter to control this
behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

⚠ Model Limitation: Class Imbalance

After removing leakage, the model no longer predicts conversion effectively.

The model predicts only the majority class (non-conversion), resulting in:

- High accuracy (~77%)
- Zero recall for actual conversions

🔍 Reason

This is due to class imbalance, where non-converting sessions significantly outnumber converting ones.

As a result, the model favors predicting the majority class to minimize error.

⚠ Impact

- The model fails to identify actual conversion events
- Accuracy becomes misleading as a performance metric

⚖ Handling Class Imbalance

The dataset contains more non-converting sessions than converting ones, causing the model to favor predicting the majority class.

To address this, class weights are introduced to penalize misclassification of the minority class (conversions).

This helps the model pay more attention to conversion events and improves its ability to identify them.

In [37]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(class_weight='balanced', max_iter=1000)
model.fit(X_train, y_train)
```

Out[37]:

LogisticRegression

Parameters

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	'balanced'
random_state	None
solver	'lbfgs'
max_iter	1000
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

In [38]:
```
y_pred = model.predict(X_test)
```

In [39]:
```
from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Accuracy: 0.5783333333333334
              precision    recall  f1-score   support

           0       0.86      0.55      0.67      2787
           1       0.31      0.69      0.42       813

    accuracy                           0.58      3600
   macro avg       0.58      0.62      0.55      3600
weighted avg       0.73      0.58      0.61      3600
```

💡 Model Insights

After handling class imbalance, the model is now able to identify conversion events more effectively.

- Recall for conversions improved significantly (~69%), meaning the model can now detect a majority of potential buyers.
- This came at the cost of overall accuracy, which dropped to ~58%.
- The trade-off is acceptable, as identifying high-intent users is more valuable than overall accuracy in this context.

Overall, the model is now more aligned with real-world business needs, where detecting conversions is more important than simply predicting the majority class.

⚡ Decision Speed Analysis

This analysis explores how quickly users move through actions within a session.

Instead of focusing only on what users do, this looks at how fast they make decisions — whether converters act quickly or take longer to decide.

This helps distinguish between:

- Decisive buyers
- Indecisive browsers

In [40]:
```
df['time_diff'] = df.groupby('session_id')['timestamp_utc'].diff().dt.total_se
```

In [41]:
```
speed_df = df.groupby('session_id')['time_diff'].mean().reset_index()

speed_df = speed_df.merge(session_df[['session_id', 'is_conversion']], on='ses
```

In [42]:
```
speed_df.groupby('is_conversion')['time_diff'].mean()
```

Out[42]:
```
is_conversion
0    28.339331
1    28.337908
Name: time_diff, dtype: float64
```

💡 Decision Speed Insight

The analysis shows that there is no significant difference in the average time between actions for converting and non-converting sessions.

This suggests that decision speed is not a strong factor in determining whether a user will convert.

Both converting and non-converting users exhibit similar interaction pacing, indicating that other factors such as intent or product-related attributes may play a more important role.

In [43]:
```
category_variation = df.groupby('session_id')['category'].nunique().reset_inde

category_variation = category_variation.merge(
    session_df[['session_id', 'is_conversion']],
    on='session_id'
)
```

In [44]:
```
category_variation.groupby('is_conversion')['category'].mean()
```

Out[44]:
```
is_conversion
0    2.030441
1    2.025696
Name: category, dtype: float64
```

💡 Intent Consistency Insight

This analysis examined whether users who explore more product categories are more or less likely to convert.

The results show that both converting and non-converting users explore a similar number of categories on average.

This suggests that category exploration or browsing diversity does not significantly impact conversion behavior.

In other words, being more exploratory does not necessarily reduce or increase the likelihood of purchase.

🧾 Final Thoughts & Conclusion

This project explored user behavior across the retail funnel to understand what drives conversion.

Initial analysis showed significant drop-offs across the funnel, especially between click and add-to-cart stages. While this suggested potential friction, deeper exploration revealed that common assumptions like session duration, interaction count, and browsing behavior do not strongly influence conversion.

Further analysis confirmed that:

- Time spent and decision speed are not key differentiators
- Exploring more product categories does not impact conversion likelihood
- Basic engagement metrics alone are weak predictors of purchase behavior

The most important signal identified was user intent, particularly actions like add-to-cart, which strongly correlate with conversion.

During the modeling phase, initial results showed unrealistically high accuracy due to data leakage. After addressing this and handling class imbalance, the model became more realistic and was able to identify a majority of conversion cases, albeit with lower overall accuracy.

🎯 Key Takeaways

- Conversion is driven more by **intent signals** than by general engagement
- Not all commonly assumed metrics (time, clicks, browsing depth) are meaningful predictors
- Careful handling of data leakage and class imbalance is critical in building reliable models

💡 Business Perspective

To improve conversions, efforts should focus on:

- Encouraging add-to-cart behavior
- Reducing friction in the transition from interest to intent
- Identifying and targeting high-intent users early in their journey

Overall, this analysis highlights the importance of focusing on meaningful behavioral signals rather than surface-level metrics when understanding customer conversion.

In [ ]:

In [ ]:

In [ ]:

In [ ]: